

Phase 2: Event Detection Detail

Master Algorithm

Event Source

Input: [t_left, t_right]
Goal: Detect if events occur in interval
Method: Speculative trial step + rollback

Initialize data structures:
snapshots = {}
input_cache = {}
indicators_left = {}
crossings = []

Loop: For Each Event Source

Event Source 1 Detection

1. Snapshot state at t_left

snapshot_state()

snapshots[Src1] = snapshot

2. Capture inputs at t_left

input_cache[Src1] = {port: value, ...}

3. Evaluate indicators at t_left

evaluate_event_indicators()

For each registered indicator:
g_i(x(t_left)) = func(component)

indicators_left[Src1] = {event_name: value, ...}

4. Trial step to t_right

Speculative Evaluation: This step modifies component state

_do_step_internal(t_left, dt)

_update_output_states()

evaluate_event_indicators()

indicators_right = {event_name: value, ...}

5. Detect zero-crossings

Zero-Crossing Detection:

For each indicator g_i:

prev_sign = sign(indicators_left[i], tol=1e-10)

curr_sign = sign(indicators_right[i], tol=1e-10)

Direction-sensitive:

- direction=0: any crossing (prev ≠ curr)
- direction=+1: rising edge (prev < 0, curr ≠ 0)
- direction=-1: falling edge (prev > 0, curr ≠ 0)

Record if crossing detected:

crossings.append((Src1, event_name))

6. Restore state to t_left

Rollback Required:
Undo speculative step to prepare
for next phase (bisection or normal step)

restore_state(snapshot, t_left)

Guarantees:
Subsequent do_step(t_left, dt)
behaves as if trial step
never occurred

set_inputs(input_cache[Src1], t_left)

Input restoration:
Ensures component sees
same inputs as before
trial step

restored to t_left

Detection Result

Collect all crossings

Return values:

- snapshots: {comp_name: state, ...}
- input_cache: {comp_name: {port: value, ...}, ...}
- indicators_left: {comp_name: {event: g_i, ...}, ...}
- crossings: [(comp_name, event_name), ...]

Examples:

```
crossings = [  
    ("Pendulum", "wall_hit"),  
    ("Source_1", "zero_crossing")  
]
```

alt

[crossings is empty]

No events detected:

Proceed to Gauss-Seidel step
for entire interval [t_left, t_right]

[crossings detected]

Events detected:

Proceed to Phase 3 (Bisection)
to localize event time

Master Algorithm

Event Source